

COVID-19 Case Data (larger, time-series, good "big data" feel) —

<https://www.kaggle.com/datasets/imdevskp/corona-virus-report>

for based on given syllabus give best easy topic for assignment and hadoop and no sql is mandatory other like spark, mapreduce is optional choose any one for extra give we have 3 member team give topic name and work if dataset required then give link also

give big data for covid 19 give link

Yes. If your syllabus makes **HDFS + Hadoop mandatory**, while **MapReduce/Spark are optional**, I would keep the project simple and focus on **HDFS + Hadoop + NoSQL**, then add **one optional technology** such as MapReduce for extra marks.

Best topic for your 3-member team

“COVID-19 Global Case Analysis using Hadoop HDFS and NoSQL”

This is a very suitable Big Data assignment because the COVID dataset is **time-series + country-wise + large enough to demonstrate distributed storage and processing**.

Your given Kaggle dataset is a good choice:

COVID-19 Corona Virus Report — Kaggle

The dataset is also referenced by projects that use it for global COVID dashboards and time-series analysis.  GitHub +1

Recommended project structure

Main technologies

Technology	Requirement	Use in project
HDFS	Mandatory	Store COVID CSV datasets in distributed storage
Hadoop	Mandatory	Hadoop ecosystem/environment

Technology	Requirement	Use in project
NoSQL	Mandatory	Store/query processed COVID records
MapReduce	Optional	Perform distributed COVID statistics
Spark	Skip	Not necessary; keeps project easier

I recommend:

HDFS + Hadoop MapReduce + MongoDB

This gives you a clear Big Data architecture without making the project unnecessarily difficult.

3-member team work division

Member 1 — HDFS + Hadoop

Responsibility: Data Storage & Hadoop Setup

Tasks:

- Install/configure Hadoop
- Configure HDFS
- Download COVID dataset
- Upload CSV files into HDFS
- Demonstrate:
 - `mkdir`
 - `put`
 - `ls`
 - `cat`
 - `du`
 - `get`
- Explain NameNode and DataNode
- Prepare screenshots for the report

Example:

```
COVID Dataset
  ↓
CSV Files
  ↓
```

HDFS

```
↓  
/covid19/  
├─ covid_19_data.csv  
├─ day_wise.csv  
├─ country_wise_latest.csv  
└─ time_series_covid19.csv
```

Member 2 — MapReduce

Responsibility: Big Data Processing

Implement 2–4 simple MapReduce operations.

Analysis 1: Total confirmed cases by country

Country → Total Confirmed Cases

Analysis 2: Total deaths by country

Country → Total Deaths

Analysis 3: Top 10 countries by confirmed cases

Country → Confirmed Cases

Analysis 4: Daily global cases

Date → Confirmed Cases

This is the part that gives your project the strongest **Big Data processing** feel.

You don't need complicated algorithms.

Member 3 — NoSQL + Results

Responsibility: MongoDB / NoSQL

After Hadoop/MapReduce processing, store the useful results in MongoDB.

For example:

JSON

```
{  
  "country": "India",  
  "confirmed": 45000000,  
  "deaths": 500000,  
  "recovered": 44000000  
}
```

Collections:

covid_database

- |— country_statistics
- |— daily_statistics
- |— top_countries

Then perform queries such as:

Find countries with confirmed cases > X

Find top 10 countries by deaths

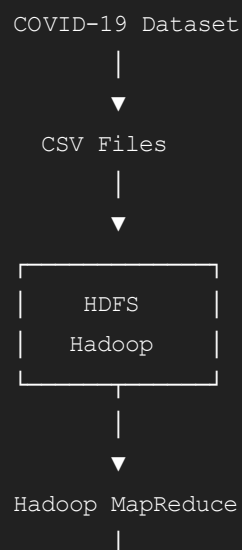
Find India's COVID statistics

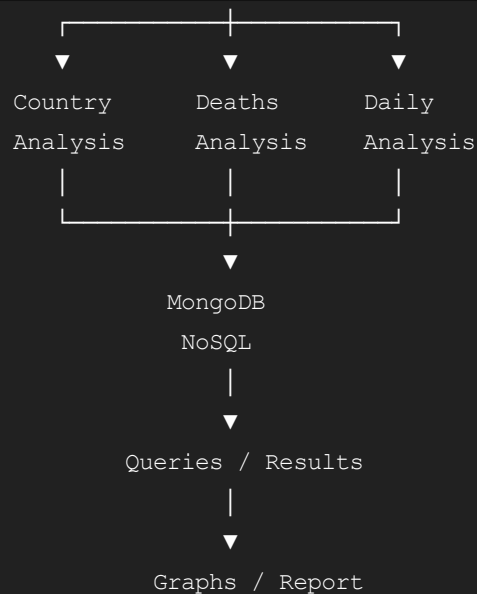
Find daily statistics for a particular date

Member 3 can also prepare simple graphs/tables from the MongoDB output.

Complete architecture

Your final project can show this architecture:





This is more than enough for a college Big Data assignment.

Dataset options

Option 1 — Recommended: Your Kaggle dataset

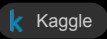
[COVID-19 Corona Virus Report — Kaggle](#)

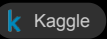
I would use this one because it gives you a good global/time-series project.

Option 2 — COVID-19 India dataset

If your professor prefers an **India-focused** project, this is actually an excellent alternative:

[COVID-19 Corona Virus India Dataset — Kaggle](#)

It contains multiple useful files including `complete.csv`, patient-level data, nation-level daily data, state-level data, district-level data, and testing data. 

The Kaggle notebook indicates the collection contains roughly **110 MB** of data across these files, which gives a more convincing Big Data demonstration than using only one tiny CSV. 

For your assignment, this could actually be **better than the global dataset** if your teacher wants you to demonstrate multiple datasets/files in HDFS.

Option 3 — Johns Hopkins original time-series data

Johns Hopkins maintains historical COVID data covering **January 22, 2020 to March 10, 2023**, including global confirmed cases and deaths time series.  [coronavirus.jhu...](#) +1

[Johns Hopkins COVID-19 Data Repository](#)

There is also a Zenodo copy containing global confirmed, deaths and recovered time-series CSV files.  [Zenodo](#)

What I would choose

For **easy implementation + good marks**, choose:

COVID-19 Global Case Analysis using Hadoop HDFS, MapReduce and MongoDB

Technology stack

```
Dataset
  ↓
HDFS
  ↓
Hadoop
  ↓
MapReduce
  ↓
MongoDB
  ↓
Analysis / Visualization
```

Why this is the best choice

- HDFS requirement → **covered**
- Hadoop requirement → **covered**
- NoSQL requirement → **covered**
- Optional MapReduce → **covered**
- Spark → **not required**
- Easy dataset → **available**
- Time-series data → **good Big Data feel**
- Three members → **easy to divide**
- Easy to demonstrate in viva
- Easy to create screenshots/results
- No complicated machine learning required

Suggested final title

“Big Data Analysis of COVID-19 Cases Using Hadoop HDFS, MapReduce and MongoDB”

If you want something slightly more impressive:

“Distributed COVID-19 Data Analysis Using Hadoop HDFS, MapReduce and NoSQL”

I recommend the **second title** for your college submission because it emphasizes the actual Big Data technologies rather than just the disease dataset.

i need big data

Basic Project Rules

Team

- Maximum **3 students per group**.
- Each student must have a clearly defined role.
- Individual contribution will be checked through **GitHub commit history**.

Project Topic

Show more

Yes — with these rules, your project should be designed as a **real distributed Big Data project**, not just “COVID analysis in Python.” The important point is that your professor requires **3-node distributed execution**, so I would choose a dataset large enough to justify HDFS and distributed processing.

Best COVID Big Data project

Project Title

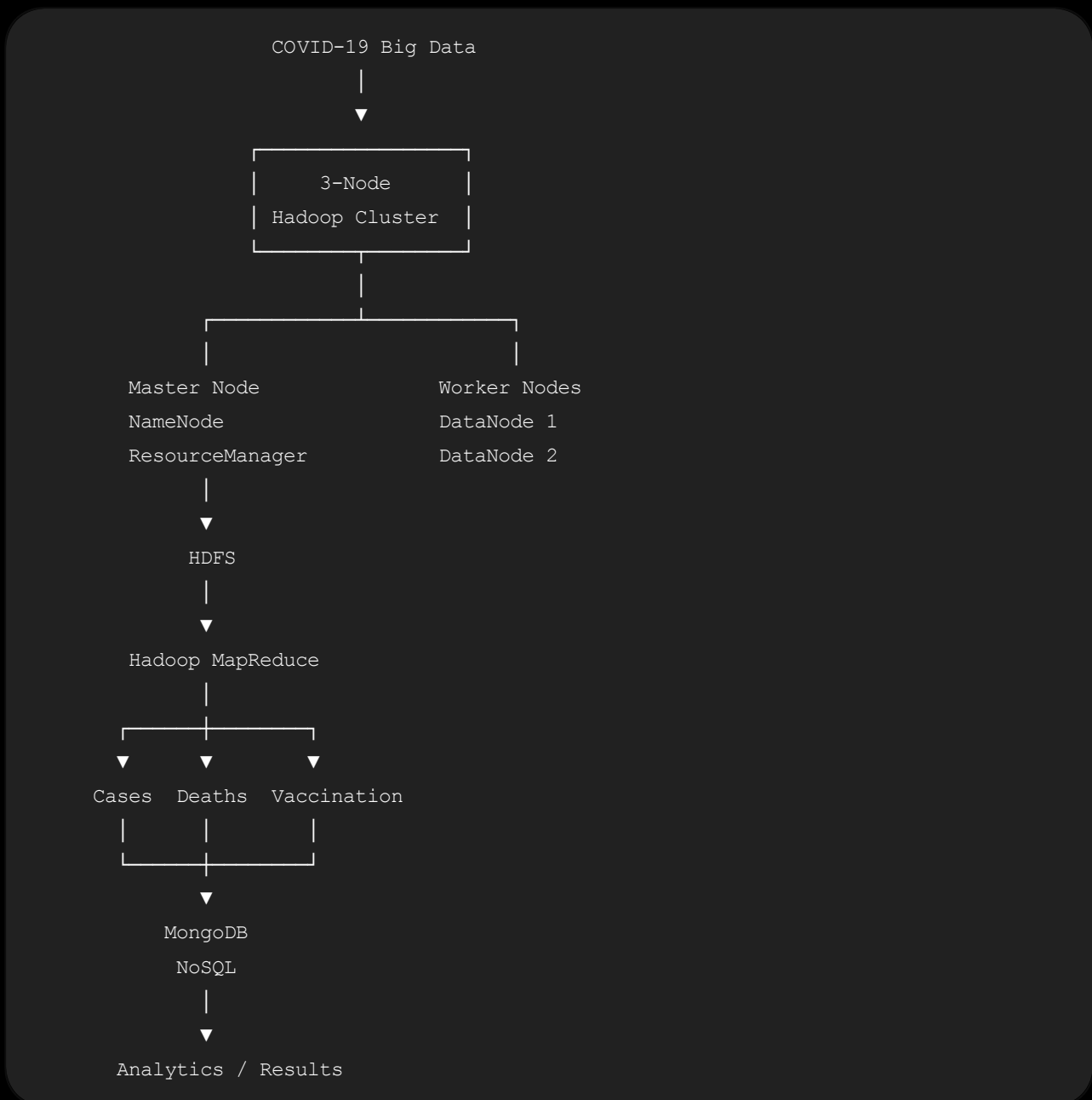
“Distributed COVID-19 Big Data Analytics using Hadoop HDFS, MapReduce and MongoDB”

Recommended stack

HDFS + Hadoop MapReduce + MongoDB

You do **not** need Spark. MapReduce is enough as the optional processing technology and is easier to demonstrate than Spark.

Your architecture:



Important: about the dataset size

The Kaggle dataset you initially mentioned:

[COVID-19 Corona Virus Report — Kaggle](#)

is **good for learning**, but I would **not use it as your only dataset** if your professor specifically expects a strong Big Data project with a 3-node cluster.

You should use a **larger COVID data collection**, preferably multiple datasets/tables, so your HDFS storage and distributed processing are meaningful.

1. Best choice — Google COVID-19 Open Data

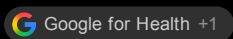
Google COVID-19 Open Data

Google COVID-19 Open Data

This is my **#1 recommendation** for your project.

Google describes it as one of the largest aggregations of COVID-related data, covering **20,000+ locations worldwide** and multiple categories including:

- Epidemiology
- Cases
- Deaths
- Tests
- Vaccination
- Hospitalization
- Demographics
- Economy
- Mobility
- Government response
- Weather
- Geography



The underlying repository is also available on GitHub:

Google COVID-19 Open Data GitHub

It provides separate large tables and an aggregated table, and the data is organized using location and date keys.  GitHub

Direct important dataset

Epidemiology time-series:

COVID Epidemiology CSV

The repository documents this as the full time-series epidemiology table.  GitHub

You can also use:

Aggregated dataset:

COVID Aggregated Data

Why I prefer this

Instead of forcing one small CSV to become “Big Data,” you can put **multiple related tables into HDFS**:

```
dataset/  
|  
├─ epidemiology.csv  
├─ demographics.csv  
├─ economy.csv  
├─ mobility.csv  
├─ government_response.csv  
├─ health.csv  
├─ hospitalizations.csv  
├─ vaccination.csv  
├─ weather.csv  
└─ index.csv
```

Then your project genuinely demonstrates:

Distributed Storage + Distributed Processing + NoSQL

2. AWS COVID-19 Data Lake

Another very strong option is the official AWS COVID-19 Data Lake.

AWS COVID-19 Data Lake

AWS describes this as a centralized public data lake containing curated COVID datasets, including:

- COVID case tracking
- Testing
- Hospital bed availability
- Health surveys
- Research literature
- Other COVID-related datasets

 Amazon Web ...

This is particularly interesting for a **Big Data** project because it is already structured as a data lake and is hosted in Amazon S3.

However

For your college project, I would choose **Google COVID Open Data over AWS** because downloading and demonstrating the data locally with Hadoop is simpler.

3. Our World in Data COVID dataset

Another good source:

[Our World in Data COVID Dataset](#)

It contains historical COVID information including:

- Confirmed cases
- New cases
- Deaths
- Testing
- Vaccination
- Hospitalization
- Population
- Population density
- Demographic variables
- Government response indicators

 Kaggle +1

The stable dataset URL is documented by the project.  Kaggle

You can also use the Kaggle mirror:

[Our World in Data COVID Dataset — Kaggle](#)

4. COVID-19 India — if you want an India-focused project

You can also use:

[COVID-19 India Dataset — Kaggle](#)

The dataset contains multiple files covering things such as:

- Patient data
- State-level data
- District-level data

- Daily data
- Testing data
- Vaccination-related information

This gives you several files to put into HDFS.

There is also the original COVID-19 India data repository:

COVID19 India Data API

It provides numerous downloadable CSV files containing raw and state/district-level COVID data.  data +1

My ranking for your assignment

Rank	Dataset	Big Data suitability	Difficulty	Recommendation
1	Google COVID Open Data	★★★★★	Medium	Best
2	AWS COVID Data Lake	★★★★★	Medium/Hard	Very good
3	COVID-19 India multiple datasets	★★★★☆	Easy/Medium	Very good
4	Our World in Data	★★★★☆	Easy	Good
5	Your original Kaggle dataset	★★☆☆☆	Very Easy	Too small alone

But you asked specifically for datasets in GBs

This is the important part.

Don't simply claim that a dataset is "GB-sized" unless you have verified the actual downloaded size. Many COVID datasets that look like Big Data are actually only MBs.

Instead, for your assignment, I recommend creating a **multi-source COVID Big Data collection** and storing all of it in HDFS.

For example:

```
COVID-BIG-DATA/  
├── epidemiology/  
├── vaccination/  
├── hospitalization/  
├── mobility/  
├── weather/  
├── demographics/  
├── economy/  
├── government_response/  
├── testing/  
└── geography/
```

Then:

```
Local Dataset  
    ↓  
HDFS  
    ↓  
3-node Hadoop Cluster  
    ↓  
HDFS blocks distributed across  
DataNode 1 + DataNode 2  
    ↓  
MapReduce  
    ↓  
MongoDB
```

That is much more defensible in your viva than taking a 50–100 MB CSV and calling it Big Data.

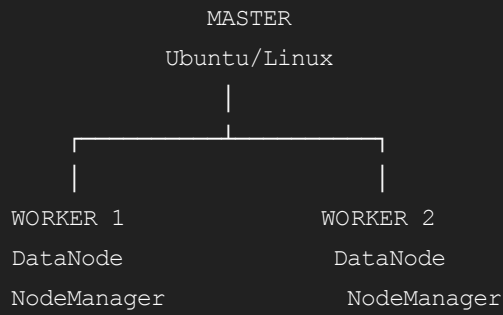
Google's dataset is particularly useful here because it combines COVID epidemiology with many covariates and more than 20,000 locations. [GitHub](#)

What your 3-node cluster should demonstrate

Your professor explicitly says:

1 Master + minimum 2 Workers

So make this your setup:



Master

```
NameNode  
ResourceManager  
SecondaryNameNode
```

Worker 1

```
DataNode  
NodeManager
```

Worker 2

```
DataNode  
NodeManager
```

Then upload the COVID dataset:

Bash

```
hdfs dfs -mkdir -p /covid19  
hdfs dfs -put epidemiology.csv /covid19/
```

Check:

Bash

```
hdfs dfsadmin -report
```

You should be able to show:

```
Live datanodes (2)
```

Then:

```
Bash
```

```
hdfs dfs -ls /covid19
```

and demonstrate the file/block distribution.

MapReduce part

Keep the MapReduce analysis simple.

Job 1 — Country-wise confirmed cases

```
Country → Total Confirmed Cases
```

Job 2 — Country-wise deaths

```
Country → Total Deaths
```

Job 3 — Daily new cases

```
Date → New Cases
```

Job 4 — Top 10 affected countries

```
Country → Cases
```

This is sufficient to demonstrate distributed processing.

MongoDB part

After MapReduce:

```
MapReduce Output
```

```
↓
```

```
MongoDB
```

```
↓
```

```
country_statistics
```

```
daily_statistics
```

```
death_statistics
```

Example document:

JSON

```
{  
  "country": "India",  
  "date": "2022-01-15",  
  "confirmed": 36500000,  
  "deaths": 485000,  
  "new_cases": 194000  
}
```

Then demonstrate MongoDB queries.

3-member division

Student 1 — Hadoop/HDFS Engineer

Role: Cluster + Distributed Storage

Work:

- 3-node cluster setup
- Hadoop configuration
- NameNode
- DataNodes
- HDFS
- Dataset ingestion
- HDFS block distribution
- Cluster report
- GitHub commits

Suggested GitHub directories:

```
config/  
scripts/cluster/  
scripts/hdfs/  
docs/cluster/  
screenshots/hdfs/
```

Student 2 — MapReduce/Data Processing Engineer

Role: Distributed Analytics

Work:

- Mapper
- Reducer
- Hadoop Streaming or Java MapReduce
- Country-wise analysis
- Daily analysis
- Top-country analysis
- Job execution
- Performance/output comparison

Directories:

```
src/processing/  
src/analytics/  
scripts/mapreduce/  
output/
```

Student 3 — NoSQL/Data Analytics Engineer

Role: MongoDB + Results

Work:

- MongoDB setup
- Create collections
- Import MapReduce output
- MongoDB queries
- Aggregation pipelines
- Result tables
- Graphs
- Final analytics

Directories:

```
src/analytics/  
scripts/mongodb/  
output/  
docs/
```

GitHub contribution plan

Your professor requires **5 meaningful commits per student**.

Don't do:

```
Student 1 → 50 files
Student 2 → nothing
Student 3 → nothing
```

Instead:

Student 1

```
commit 1: Configure Hadoop master node
commit 2: Configure worker nodes
commit 3: Add HDFS ingestion scripts
commit 4: Add cluster monitoring scripts
commit 5: Add HDFS documentation and screenshots
```

Student 2

```
commit 1: Add MapReduce mapper
commit 2: Add reducer
commit 3: Add country-wise analysis
commit 4: Add daily case analysis
commit 5: Add MapReduce execution documentation
```

Student 3

```
commit 1: Configure MongoDB
commit 2: Add MongoDB collections
commit 3: Add aggregation queries
commit 4: Add analytics/output
commit 5: Add final results documentation
```

That gives you **15+ meaningful commits** rather than artificially splitting one upload into many commits.

Final recommendation

If your professor is specifically checking **Big Data + 3-node distributed execution**, I would use:

Dataset

Google COVID-19 Open Data

[Google COVID-19 Open Data](#)

Technology

```
Hadoop
+
HDFS
+
MapReduce
+
MongoDB
```

Project

“Distributed COVID-19 Big Data Analytics using Hadoop HDFS, MapReduce and MongoDB”

Cluster

```
1 Master
+
2 Workers
```

Core demonstration

```
Large / multi-table COVID dataset
↓
HDFS
↓
3-node cluster
↓
MapReduce
↓
COVID analytics
↓
MongoDB
↓
Final results
```

One correction to your original plan: I would **not** use the original `imdevskp/corona-virus-report` dataset alone for this assignment. It is excellent for a basic COVID project, but your professor's explicit 3-node cluster and distributed-storage/processing requirements make a **larger multi-table COVID collection such as Google COVID-19 Open Data** a much stronger choice. The Google repository explicitly provides multiple tables and data at country/province/local levels, making it much easier to justify HDFS and distributed processing.  [GitHub](#)

choose google 1 dataset and generate word file for this that in our assignment

Done. I selected the **Google COVID-19 Open Data – aggregated.csv** dataset and prepared the Word assignment around your exact requirements:

- 3-node Hadoop cluster
- 1 Master + 2 Workers
- HDFS distributed storage
- Hadoop MapReduce
- MongoDB NoSQL
- 3-member work division
- GitHub structure and 5+ commits per student
- HDFS and MapReduce demonstration commands
- Expected analytics and outputs
- Official Google dataset links
- Honest note about the dataset size rather than falsely claiming it is multi-GB

The Google repository documents the aggregated table as a flat table joining data across multiple COVID-related tables and more than 20,000 global locations.  [github.com](#) +1

Download the Word file:

 [Download COVID Big Data Assignment \(.docx\)](#) 



COVID_Big_Data_Hadoop_Assignment.docx
Document

give descriptiona dn platform info that i fill in sheet

You can fill your assignment sheet with the following:

Project Title

Distributed COVID-19 Big Data Analytics using Hadoop HDFS, MapReduce and MongoDB

Project Description

This project analyzes large-scale COVID-19 time-series data using a distributed Big Data environment. The Google COVID-19 Open Data dataset will be stored in HDFS across a 3-node Hadoop cluster consisting of one Master node and two Worker nodes. Hadoop MapReduce will be used to perform distributed analytics such as country-wise confirmed cases, deaths, daily cases, and top affected locations. The processed results will then be stored and queried using MongoDB, a NoSQL database. The project demonstrates distributed storage, distributed processing, NoSQL analytics, and an end-to-end Big Data pipeline.

Dataset

Google COVID-19 Open Data – Aggregated Dataset (`aggregated.csv`)

Dataset Link

[Google COVID-19 Open Data](#)

Platform / Technologies

Hadoop Ecosystem, HDFS, Hadoop MapReduce, MongoDB, Linux/Ubuntu, GitHub

Platform Information

The project will be implemented on a 3-node Hadoop cluster with 1 Master node and 2 Worker nodes. The Master node will run NameNode and ResourceManager, while the Worker nodes will run DataNode and NodeManager. HDFS will provide distributed storage, Hadoop MapReduce will perform distributed data processing, and MongoDB will be used for NoSQL storage and querying of processed results.

Big Data Concepts Used

- **HDFS** – Distributed storage
- **Hadoop** – Cluster management
- **MapReduce** – Distributed processing
- **MongoDB** – NoSQL database
- **Data Analytics** – COVID-19 trends and statistics
- **Cluster Computing** – 1 Master + 2 Worker nodes

Expected Output

Country-wise COVID-19 cases and deaths, daily case trends, top 10 affected locations, and MongoDB-based analytical queries/results.

Team Size

3 Students

Short Description — if the sheet has limited space

A distributed COVID-19 Big Data analytics project using a 3-node Hadoop cluster. COVID-19 data is stored in HDFS, processed using Hadoop MapReduce, and analyzed/stored using MongoDB NoSQL. The project demonstrates distributed storage, processing, and analytics.

Apache Spark is useful or not in this

Yes, **Apache Spark is useful**, but for your specific assignment I would say **it is optional, not necessary**.

Your professor already allows:

- HDFS
- Hadoop
- MapReduce
- Hive
- Pig
- HBase
- MongoDB/Cassandra
- Spark
- Hadoop ecosystem

And your strongest requirement is **3-node distributed execution**.

My recommendation

For your COVID project:

HDFS + Hadoop + Spark + MongoDB

is actually a **stronger and more modern combination** than HDFS + MapReduce + MongoDB.

You can use:

```
Google COVID Dataset
  ↓
HDFS
  ↓
3-Node Hadoop Cluster
  ↓
```

Apache Spark



COVID Analytics



MongoDB

Why Spark is useful

Spark is particularly good for your dataset because you can perform:

- Country-wise confirmed cases
- Country-wise deaths
- Daily COVID trends
- Top 10 affected countries
- Group-by and aggregation
- Filtering and sorting
- DataFrame-based analytics

And most importantly, **Spark can run in distributed mode across your two worker nodes**, which helps you demonstrate the cluster requirement.

Spark vs MapReduce

Feature	MapReduce	Spark
Distributed processing	Yes	Yes
Works with HDFS	Yes	Yes
Easy for analytics	Medium	Easy
Performance	Slower	Faster
DataFrame API	No	Yes
Modern Big Data technology	Older	Yes
Easy COVID analytics	Medium	Very easy
Good for your assignment	Yes	Excellent

Best choice for your team

I would now change the project stack to:

HDFS + Hadoop + Apache Spark + MongoDB

You can **skip MapReduce**.

That makes the project easier to develop while still demonstrating genuine distributed Big Data processing.

Your final technology description can be:

Platform/Technologies:

Hadoop HDFS, Apache Spark, MongoDB, Linux/Ubuntu, GitHub

Cluster:

1 Master Node + 2 Worker Nodes

Processing:

Apache Spark running on the Hadoop cluster

Storage:

HDFS

NoSQL:

MongoDB

This is probably the **best balance of easy implementation + strong Big Data impression** for your assignment.

This project analyzes large-scale COVID-19 time-series data using a distributed Big Data environment. The Google COVID-19 Open Data dataset will be stored in HDFS across a 3-node Hadoop cluster consisting of one Master node and two Worker nodes. Hadoop MapReduce will be used to perform distributed analytics such as country-wise confirmed cases, deaths, daily cases, and top affected locations. The processed results will then be stored and queried using MongoDB, a NoSQL database. The project demonstrates distributed storage, distributed processing,

Show more

For your COVID-19 project, matching the **same short style** as the examples in your sheet, I recommend this:

Description

COVID-19-related datasets are stored and distributed across a 3-node Hadoop HDFS cluster. Apache Spark/PySpark is used for distributed data preprocessing and analytics, while MongoDB stores COVID-19 information and analytical results. The project analyzes country-wise cases, deaths, daily trends, and top affected locations.

Platform / Technologies

Hadoop HDFS (3-node cluster), Apache Spark/PySpark, MongoDB, Python, Jupyter Notebook, and GitHub.

Even shorter version for the sheet

Description:

COVID-19 datasets are stored and distributed across a 3-node Hadoop HDFS cluster. Apache Spark/PySpark is used for distributed data preprocessing and analytics, while MongoDB stores COVID-19 data and analysis results.

Platform:

Hadoop HDFS (3-node cluster), Apache Spark/PySpark, MongoDB, Python, Jupyter Notebook, and GitHub.

This version is better than your earlier MapReduce description because it clearly shows **HDFS + 3-node cluster + Spark + NoSQL**, which gives your project a stronger Big Data profile.

give good heading

Recommended Project Heading

Distributed COVID-19 Big Data Analytics using Hadoop HDFS, Apache Spark and MongoDB

Other good options:

- 1. COVID-19 Big Data Analytics using Hadoop, Apache Spark and MongoDB**
- 2. Distributed COVID-19 Data Processing and Analytics using Hadoop HDFS and Spark**
- 3. Scalable COVID-19 Big Data Analysis using Hadoop HDFS, Spark and MongoDB**
- 4. Distributed COVID-19 Case Analysis using Hadoop HDFS, Apache Spark and NoSQL**

5. COVID-19 Time-Series Big Data Analytics using Hadoop HDFS, Spark and MongoDB

Best for your assignment sheet:

Distributed COVID-19 Big Data Analytics using Hadoop HDFS, Apache Spark and MongoDB

give brief info how make project for explaining it

You can explain the project to your faculty/team member like this:

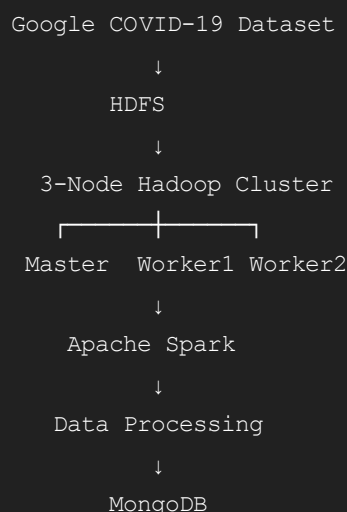
Project Explanation

Our project is a Distributed COVID-19 Big Data Analytics system. We will use the Google COVID-19 Open Data dataset and process it on a **3-node Hadoop cluster** consisting of **1 Master node and 2 Worker nodes**.

First, the COVID-19 dataset will be uploaded to **HDFS**, where the data will be distributed across the worker nodes. Then, **Apache Spark/PySpark** will run on the cluster to process the data in parallel. We will perform analytics such as **country-wise confirmed cases, deaths, daily case trends, and top affected countries/locations**.

After processing, the important results will be stored in **MongoDB**, which will be used as our NoSQL database for querying and retrieving the analytical results. **Python and Jupyter Notebook** can be used for analysis and visualization, while **GitHub** will be used for version control and showing each team member's contribution.

Simple Flow



What we will actually demonstrate

1. Create 3-node cluster

- 1 Master
- 2 Workers

2. Configure Hadoop HDFS

- Start NameNode/DataNodes
- Check cluster status
- Upload COVID dataset

3. Run Spark

- Read data from HDFS
- Clean/preprocess data
- Perform distributed operations

4. Perform analytics

- Total cases by country
- Total deaths by country
- Daily cases
- Top 10 affected countries
- Case/death trends

5. Use MongoDB

- Store processed results
- Query results
- Perform aggregation

6. Show proof of Big Data

- 3-node cluster status
- HDFS storage
- Spark job execution
- Worker-node processing
- MongoDB results

One-line explanation for viva

“We are taking COVID-19 time-series data, storing it in distributed HDFS across a 3-node Hadoop cluster, processing it in parallel using Apache Spark, and storing the analytical results in MongoDB for NoSQL querying.”

This is a good project because **HDFS demonstrates distributed storage, Spark demonstrates distributed processing, and MongoDB demonstrates NoSQL**, so all the major Big Data components are clearly visible.

Today 6:16 PM

now i start my project group of three students and i am leader and main developer also and ma'am said they also see github commit history so give proper roadmap for this project

1. Basic Project Rules

Team

- Maximum **3 students per group**.
- Each student must have a clearly defined role.
- Individual contribution will be checked through **GitHub**

Show more

Yes. Given your faculty's rules and your goal of making this **resume-worthy and industry-oriented**, I would build it as a small **production-style distributed data platform**, not just a notebook that analyzes COVID data.

One important clarification: Google's COVID-19 Open Data repository is now archived/read-only, so this should be presented as a **historical COVID-19 analytics project**, not a live COVID monitoring system. The dataset itself is substantial in scope: it combines daily time-series data across 20,000+ locations and includes epidemiology, hospitalizations, vaccinations, mobility, demographics, weather, government response and other tables.  [GitHub](#)

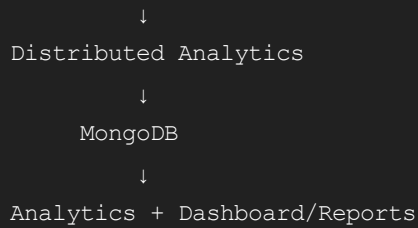
1. Final project definition

Project title

Distributed COVID-19 Big Data Analytics Platform using Hadoop HDFS, Apache Spark and MongoDB

Core stack

```
Google COVID-19 Open Data
    ↓
Data Ingestion
    ↓
Hadoop HDFS Cluster
┌───┴───┐
Master Worker-1 Worker-2
    ↓
Apache Spark / PySpark
```



Mandatory technologies

- Hadoop HDFS
- 3-node cluster
- Apache Spark / PySpark
- MongoDB
- Python
- Jupyter Notebook
- GitHub

Additional technologies I recommend

Don't add everything just to make the project look complicated. Add technologies that have a clear purpose:

- **Docker** → reproducible 3-node cluster
- **Docker Compose** → start/stop cluster easily
- **PySpark** → distributed processing
- **MongoDB** → serving analytical results
- **Pandas** → small-scale validation only
- **Matplotlib/Plotly** → visualizations
- **pytest** → basic data-pipeline testing
- **GitHub Actions** → automated testing/linting
- **MLlib** → optional advanced analytics later

2. The unique part I recommend

Don't make your project simply:

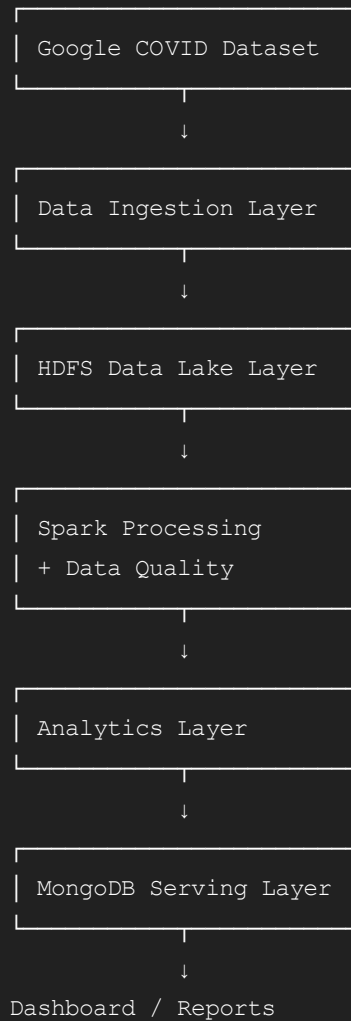
"COVID data → Spark → graphs"

That is too basic for a resume.

Instead make it:

Distributed COVID-19 Data Engineering & Analytics Pipeline

The pipeline should have **five layers**:



That gives you an actual **data engineering architecture**.

3. Dataset strategy

I recommend that you **do not download only one CSV**.

Google's project provides separate tables such as:

- `epidemiology.csv`
- `hospitalizations.csv`
- `vaccinations.csv`
- `mobility.csv`
- `demographics.csv`
- `health.csv`
- `weather.csv`
- `government_response.csv`
- `index.csv`

- `aggregated.csv`

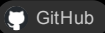
The repository explicitly documents these tables and their common `key / date` structure.



For your first version, use:

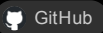
Primary

`epidemiology.csv`

It contains date-region COVID measures such as new confirmed, deceased, recovered and tested values. 

Secondary

`vaccinations.csv`

Useful for vaccination analysis. 

Third

`hospitalizations.csv`

Useful for hospitalization/ICU/ventilator analysis. 

Dimension

`index.csv`

Use it to map location keys to country/region information.

This is much better than blindly putting every available table into your project.

4. Three-person team structure

Since **you are the leader + main developer**, don't make the mistake of doing 80% of the work while your teammates only write documentation.

Your architecture should naturally divide into three technical ownership areas.

YOU — Lead Data Engineer / System Architect

Ownership

Architecture + HDFS + Spark + Integration

You should own:

```
HDFS
Spark
Pipeline architecture
Integration
Performance
Final deployment
```

Your responsibilities:

- Design architecture
- Configure Hadoop cluster
- Configure HDFS
- Build Spark pipeline
- Integrate HDFS → Spark → MongoDB
- Performance testing
- Final integration
- Code review
- GitHub repository management

This makes your contribution clearly visible.

Member 2 — Cluster & Data Engineering

Ownership

Cluster infrastructure + ingestion + data quality

Responsibilities:

- Worker node setup
- Hadoop configuration
- HDFS ingestion scripts
- Dataset download
- Data validation
- HDFS directory structure
- HDFS replication/block demonstration
- Cluster monitoring
- Docker/VM setup documentation

They should own:

```
config/
scripts/cluster/
```

```
scripts/ingestion/  
src/ingestion/  
docs/cluster/
```

Member 3 — Analytics & NoSQL

Ownership

MongoDB + Analytics + Visualization

Responsibilities:

- MongoDB schema
- MongoDB ingestion
- Aggregation queries
- Analytical datasets
- Visualizations
- Jupyter notebooks
- Result documentation
- Dashboard/report

They should own:

```
src/analytics/  
src/mongodb/  
notebooks/  
output/  
docs/analytics/
```

5. GitHub architecture

Create the repository **before starting development**.

For example:

```
covid-big-data-analytics/  
|  
├─ README.md  
├─ LICENSE  
├─ .gitignore  
├─ requirements.txt  
|  
├─ dataset/
```

```
|   └─ README.md
|
├─ src/
|   ├── ingestion/
|   ├── processing/
|   ├── analytics/
|   └─ mongodb/
|
├─ spark/
|   ├── jobs/
|   └─ utils/
|
├─ scripts/
|   ├── cluster/
|   ├── hdfs/
|   ├── ingestion/
|   └─ deployment/
|
├─ config/
|   ├── hadoop/
|   ├── spark/
|   └─ mongodb/
|
├─ notebooks/
|
├─ tests/
|
├─ output/
|
├─ screenshots/
|
├─ docs/
|   ├── architecture/
|   ├── cluster/
|   ├── analytics/
|   └─ report/
|
├─ pipeline/
|
└─ docker/
    ├── master/
    ├── worker1/
    └─ worker2/
```

Very important

Do **not** put the entire raw dataset in GitHub.

Add it to `.gitignore`.

```
dataset/raw/
output/raw/
```

```
*.csv  
*.parquet  
*.json
```

Put the download instructions and dataset URLs in:

```
dataset/README.md
```

6. Phase 1 — Repository setup

Do this **first**, before installing everything.

Step 1

Create GitHub repository.

Step 2

Add all three students as collaborators.

Step 3

Create the directory structure.

Step 4

Create:

```
README.md  
.gitignore  
LICENSE  
requirements.txt
```

Step 5

Create a project board/issues.

For example:

```
#1 Setup repository  
#2 Configure Master  
#3 Configure Worker 1  
#4 Configure Worker 2  
#5 HDFS ingestion  
#6 Spark setup  
#7 Spark analytics  
#8 MongoDB integration  
#9 Visualization
```

#10 Testing

#11 Final documentation

This makes your project look much more professional.

7. Phase 2 — Set up the 3-node cluster

You have two practical choices.

Option A — Docker

Recommended for reproducibility.

```
Docker Host
|
├─ master
|   ├── NameNode
|   └── ResourceManager
|
├─ worker1
|   ├── DataNode
|   └── NodeManager
|
└─ worker2
    ├── DataNode
    └── NodeManager
```

You can start everything with:

Bash

```
docker compose up -d
```

This is excellent for a GitHub project because another person can reproduce your environment.

Option B — Three VMs

```
VM1 → Master
VM2 → Worker 1
VM3 → Worker 2
```

Ubuntu VMs are also fine.

My recommendation

Use:

Docker + Hadoop + Spark

if your laptops can handle it.

Otherwise:

3 Ubuntu VMs

8. Phase 3 — Verify Hadoop

Before touching Spark, prove that HDFS works.

Master:

Bash

```
jps
```

You should see relevant Hadoop services.

Then:

Bash

```
hdfs dfsadmin -report
```

Your final demonstration should show:

```
Live datanodes (2)
```

Then:

Bash

```
hdfs dfs -mkdir -p /covid/raw
```

Upload dataset:

Bash

```
hdfs dfs -put epidemiology.csv /covid/raw/
```

Check:

Bash

```
hdfs dfs -ls /covid/raw
```

Then the most important command:

Bash

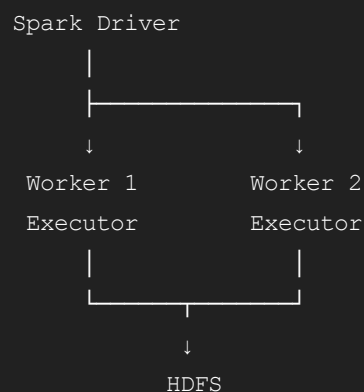
```
hdfs fsck /covid/raw/epidemiology.csv -files -blocks -locations
```

This helps demonstrate where HDFS blocks are stored.

9. Phase 4 — Spark cluster

Now configure Spark to use the cluster.

Architecture:



Your Spark job should read:

```
hdfs:///covid/raw/epidemiology.csv
```

not:

```
C:/Users/Fenil/Desktop/covid.csv
```

This distinction is important.

The latter is a local Python project.

The former demonstrates **distributed Big Data processing**.

10. Phase 5 — Build your Spark pipeline

Start simple.

Step 1 — Read

Python

Run

```
df = spark.read.csv(
    "hdfs:///covid/raw/epidemiology.csv",
    header=True,
    inferSchema=True
)
```

Step 2 — Data cleaning

Handle:

- null values
- invalid dates
- duplicate records
- incorrect numeric types
- missing location keys

Step 3 — Transform

Create useful columns:

```
date
country
region
new_cases
new_deaths
new_recovered
```

Step 4 — Analytics

Implement:

Analytics 1

Global daily cases

```
date → total new cases
```

Analytics 2

Country-wise cases

```
country → total cases
```

Analytics 3

Country-wise deaths

```
country → total deaths
```

Analytics 4

Top 10 affected countries

Analytics 5

COVID growth trends

11. Make it more unique

After the basic analytics works, add one more meaningful feature.

I recommend:

COVID Risk / Severity Analytics

Create a derived indicator based on available epidemiological measures.

For example:

```
case_fatality_rate =  
deaths / confirmed_cases × 100
```

Then categorize:

```
Low  
Moderate  
High
```

Be careful to call it a **derived analytical indicator**, not a medical prediction.

This gives you something more interesting than simple `groupBy()`.

12. Add vaccination analytics

Using Google's vaccination table, you can analyze:

```
Country
  ↓
Vaccinated population
  ↓
Fully vaccinated population
  ↓
Vaccination rate
```

The Google dataset explicitly provides vaccination measures including newly vaccinated and cumulative vaccinated values. [GitHub](#)

This gives you another Spark job.

13. Add hospitalization analytics

The hospitalization table includes measures such as hospitalized patients, ICU patients and ventilator patients. [GitHub](#)

You can create:

```
Country
  ↓
Hospitalizations
ICU admissions
Ventilator patients
```

This makes your project considerably richer.

14. MongoDB architecture

Don't dump the entire raw dataset into MongoDB.

Use MongoDB as the **serving layer for processed results**.

For example:

```
MongoDB
|
├─ country_summary
├─ daily_summary
├─ vaccination_summary
├─ hospitalization_summary
└─ top_affected_regions
```

Example:

JSON

```
{  
  "country": "India",  
  "date": "2021-05-01",  
  "new_cases": 392488,  
  "new_deaths": 3689  
}
```

Then demonstrate MongoDB:

```
find()  
sort()  
limit()  
aggregate()  
group()  
match()
```

15. Jupyter Notebook

Use notebooks for **exploration and visualization**, not as your core processing system.

Good:

```
notebooks/  
├─ 01_data_exploration.ipynb  
├─ 02_spark_validation.ipynb  
├─ 03_covid_analytics.ipynb  
└─ 04_results_visualization.ipynb
```

The actual Spark processing should live in:

```
spark/jobs/
```

For example:

```
daily_cases.py  
country_analysis.py  
vaccination_analysis.py  
hospitalization_analysis.py
```

That distinction makes the project much more industry-friendly.

16. Testing

Add a `tests/` directory.

For example:

```
tests/  
├─ test_ingestion.py  
├─ test_transformations.py  
├─ test_analytics.py  
└─ test_mongodb.py
```

Test things such as:

```
Does the pipeline load the expected columns?  
Are dates valid?  
Are negative case values handled?  
Does country aggregation return expected output?  
Can MongoDB store the processed records?
```

This is one of the easiest ways to make your project look more professional.

17. GitHub commit strategy

This is **very important for you** because faculty explicitly checks commit history.

Don't create five meaningless commits like:

```
update  
update2  
final  
final2  
done
```

Each commit should represent actual development.

Your commits

As leader:

1. Initialize project architecture
2. Add Hadoop cluster configuration
3. Implement HDFS ingestion pipeline
4. Add Spark cluster configuration
5. Implement epidemiology Spark pipeline
6. Add country-wise analytics
7. Add vaccination analytics

8. Add hospitalization analytics
9. Integrate MongoDB
10. Add pipeline orchestration
11. Add tests
12. Optimize Spark transformations
13. Add documentation
14. Final integration

You will have **far more than five meaningful commits**.

18. Member 2 commits

1. Add worker node configuration
2. Add HDFS directory initialization
3. Add dataset download script
4. Add HDFS ingestion script
5. Add data validation module
6. Add cluster monitoring script
7. Add HDFS block verification
8. Add infrastructure documentation

19. Member 3 commits

1. Add MongoDB configuration
2. Add MongoDB schema
3. Add Spark-to-MongoDB loader
4. Add aggregation queries
5. Add COVID visualization notebook
6. Add vaccination visualization
7. Add hospitalization visualization
8. Add analytics documentation

This clearly proves that everyone contributed.

20. Your development order

Do **not** try to build everything simultaneously.

Follow this exact sequence:

PHASE 1
GitHub

```
↓
PHASE 2
3-node cluster

↓
PHASE 3
Hadoop/HDFS

↓
PHASE 4
Dataset ingestion

↓
PHASE 5
Spark cluster

↓
PHASE 6
Spark ETL

↓
PHASE 7
COVID analytics

↓
PHASE 8
MongoDB

↓
PHASE 9
Visualization

↓
PHASE 10
Testing

↓
PHASE 11
Performance optimization

↓
PHASE 12
Documentation + presentation
```

21. Final demonstration to faculty

Your final demo should follow this order:

1. Architecture

Show:

```
Master
Worker 1
Worker 2
```

2. Cluster

Run:

Bash

```
hdfs dfsadmin -report
```

Show 2 live DataNodes.

3. HDFS

Show:

Bash

```
hdfs dfs -ls /covid/raw
```

4. Block distribution

Show:

Bash

```
hdfs fsck ... -files -blocks -locations
```

5. Spark

Submit the Spark job.

Show:

Spark Driver

↓

Executor 1

Executor 2

6. Analytics

Show:

Top countries

Daily cases

Deaths

Vaccination

Hospitalization

7. MongoDB

Show collections and queries.

8. Results

Show Jupyter visualizations/dashboard.

9. GitHub

Finally show:

```
Student 1 → commits  
Student 2 → commits  
Student 3 → commits
```

This directly addresses **every requirement your faculty gave you**.

22. Resume-level version

Once completed, your project can be described much more strongly than simply "COVID analysis."

Resume project name

Distributed COVID-19 Big Data Analytics Platform

Resume bullet direction

Built a 3-node distributed Big Data pipeline using Hadoop HDFS and Apache Spark to process Google COVID-19 time-series data across 20K+ global locations, implementing distributed ETL and epidemiological analytics.

Designed a NoSQL serving layer with MongoDB for country-level, daily, vaccination and hospitalization analytics, with automated ingestion, validation and Spark-based transformations.

Containerized the distributed environment and implemented reproducible cluster setup, data-quality checks, analytical pipelines and Git-based collaborative development.

Only claim **Docker/containerization, automated ingestion, tests, etc. if you actually implement them**.

23. Your first milestone

Don't start with Spark analytics yet.

Milestone 1: Repository + 3-node cluster

Your immediate target should be:

```
[ ] GitHub repository created
[ ] 3 collaborators added
[ ] Folder structure created
[ ] .gitignore created
[ ] README created
[ ] Master node running
[ ] Worker 1 running
[ ] Worker 2 running
[ ] Hadoop installed/configured
[ ] HDFS running
[ ] 2 DataNodes visible
[ ] First meaningful commit from each member
```

Once this works, **then move to HDFS dataset ingestion.**

The official Google repository provides the dataset tables and documents the common location/date structure, so we can build the ingestion layer around those keys rather than treating the data as an arbitrary CSV.  [GitHub](#) +1

Recommended final stack

Core:

Hadoop HDFS + Apache Spark/PySpark + MongoDB

Infrastructure:

Docker + Docker Compose

Development:

Python + Jupyter + Git/GitHub

Quality:

pytest + GitHub Actions

Optional advanced component:

Spark MLlib — only after the core distributed pipeline is working.

Do not add MLlib initially. Your first goal is to get **3-node HDFS → Spark → MongoDB** working end-to-end. That is the foundation on which the resume-level features should be built.

okk now one by one phase start how to intialaize github
repo give step by step with exampple one by one phase

Absolutely. We'll do this **one phase at a time** and keep the GitHub history clean so your faculty can clearly see each member's contribution.

Phase 1 — Initialize the GitHub Repository

Goal of Phase 1

At the end of this phase, you should have:

```
GitHub Repository
├──
├── README.md
├── .gitignore
├── requirements.txt
├── dataset/
├── src/
├── spark/
├── scripts/
├── config/
├── notebooks/
├── tests/
├── output/
├── screenshots/
├── docs/
└── pipeline/
```

And **all 3 students** will have made meaningful commits.

Step 1 — Decide the repository name

I recommend:

```
covid-big-data-analytics
```

GitHub URL will look like:

```
https://github.com/<your-username>/covid-big-data-analytics
```

Project name

Distributed COVID-19 Big Data Analytics Platform

Stack

Hadoop HDFS
Apache Spark / PySpark
MongoDB
Python
Jupyter Notebook
Docker
GitHub

Step 2 — Create the GitHub repository

Go to:

[GitHub](#)

Sign in with **your own GitHub account**.

Click:

+ → New repository

Enter:

Repository name:
covid-big-data-analytics

Description:

Distributed COVID-19 Big Data Analytics using Hadoop HDFS, Apache Spark and MongoDB

Select:

Public

For this academic project, Public is generally easier for faculty to inspect.

Important

Don't upload the dataset directly during repository creation.

Select:

☒ Add a README file

For `.gitignore`, you can choose **Python**, but we'll customize it later.

License:

MIT License

Then click:

Create repository

Step 3 — Add your two teammates

Open your repository.

Go to:

Settings → **Collaborators** → **Add people**

Search your teammates' GitHub usernames.

For example:

```
fenil412
student2
student3
```

Both teammates should accept the invitation.

Final structure

```
Owner:
You — Project Leader / Lead Developer

Collaborator:
Student 2

Collaborator:
Student 3
```

This is important because faculty wants to see contribution from **three different GitHub accounts**.

Step 4 — Clone the repository

On your computer, open:

PowerShell / Git Bash

Run:

Bash

```
git clone https://github.com/YOUR_USERNAME/covid-big-data-analytics.git
```

Example:

Bash

```
git clone https://github.com/Fenil412/covid-big-data-analytics.git
```

Then:

Bash

```
cd covid-big-data-analytics
```

Check:

Bash

```
git status
```

You should see something similar to:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Step 5 — Create the project folders

Inside the repository, create:

```
covid-big-data-analytics/
|
├─ README.md
├─ .gitignore
├─ requirements.txt
|
```

```
|─ dataset/
|
|─ src/
|   |─ ingestion/
|   |─ processing/
|   |─ analytics/
|   └─ mongodb/
|
|─ spark/
|   |─ jobs/
|   └─ utils/
|
|─ scripts/
|   |─ cluster/
|   |─ hdfs/
|   |─ ingestion/
|   └─ deployment/
|
|─ config/
|   |─ hadoop/
|   |─ spark/
|   └─ mongodb/
|
|─ notebooks/
|
|─ tests/
|
|─ output/
|
|─ screenshots/
|
|─ docs/
|   |─ architecture/
|   |─ cluster/
|   |─ analytics/
|   └─ report/
|
└─ pipeline/
```

On Windows PowerShell

From inside the repository:

PowerShell

```
mkdir dataset
mkdir src
mkdir src\ingestion
mkdir src\processing
mkdir src\analytics
mkdir src\mongodb
mkdir spark
```

```
mkdir spark\jobs
mkdir spark\utils
mkdir scripts
mkdir scripts\cluster
mkdir scripts\hdfs
mkdir scripts\ingestion
mkdir scripts\deployment
mkdir config
mkdir config\hadoop
mkdir config\spark
mkdir config\mongodb
mkdir notebooks
mkdir tests
mkdir output
mkdir screenshots
mkdir docs
mkdir docs\architecture
mkdir docs\cluster
mkdir docs\analytics
mkdir docs\report
mkdir pipeline
```

Step 6 — Create `.gitignore`

This is **very important**.

Your COVID dataset may become large, and you don't want raw data inside GitHub.

Create:

```
.gitignore
```

Put:

```
gitignore

# Python
__pycache__/*
*.py[cod]
*.pyo

# Virtual environment
venv/
.env/
env/

# Jupyter
.ipynb_checkpoints/
```

```
# Large datasets
dataset/raw/
dataset/processed/
*.csv
*.parquet
*.json

# Output files
output/raw/
output/processed/

# Logs
*.log

# IDE
.vscode/
.idea/

# OS
.DS_Store
Thumbs.db

# Environment/secrets
.env
*.pem
*.key

# Spark
spark-warehouse/
metastore db/
```

Important exception

Don't ignore:

```
dataset/README.md
```

We'll put dataset download information there.

Step 7 — Create `requirements.txt`

Create:

```
requirements.txt
```

Initially:

```
pyspark
pymongo
pandas
numpy
matplotlib
plotly
jupyter
pytest
```

Later, we'll pin exact versions after the cluster environment is decided.

Step 8 — Create the main README

Your `README.md` should initially contain:

Markdown

```
# Distributed COVID-19 Big Data Analytics Platform
```

```
A distributed Big Data analytics platform for processing historical
COVID-19 data using Hadoop HDFS, Apache Spark and MongoDB.
```

```
## Project Overview
```

```
This project processes Google COVID-19 Open Data using a distributed
3-node Hadoop cluster consisting of one Master node and two Worker nodes.
```

```
The system uses:
```

- Hadoop HDFS for distributed storage
- Apache Spark / PySpark for distributed processing
- MongoDB for NoSQL storage and querying
- Python for application development
- Jupyter Notebook for exploration and visualization
- Docker for reproducible cluster deployment
- GitHub for version control and collaboration

```
## Architecture
```

```
Google COVID-19 Open Data
```

```
↓
```

```
Data Ingestion
```

```
↓
```

```
Hadoop HDFS
```

```
↓
```

```
3-Node Cluster
```

```
↓
```

```
Apache Spark
```

```
↓
Distributed Analytics
↓
MongoDB
↓
Visualization / Reports
```

Team

```
| Member | Role |
|---|---|
| Member 1 | Project Leader / Lead Data Engineer |
| Member 2 | Cluster & Data Engineer |
| Member 3 | Analytics & NoSQL Engineer |
```

Dataset

Google COVID-19 Open Data:

<https://github.com/GoogleCloudPlatform/covid-19-open-data>

Technologies

- Hadoop HDFS
- Apache Spark / PySpark
- MongoDB
- Python
- Jupyter Notebook
- Docker
- GitHub

Don't put your entire final README now. We'll improve it phase by phase.

Step 9 — Create dataset README

Inside:

```
dataset/
```

create:

```
README.md
```

Put:

Markdown

```
# Dataset
```

```
## Source

Google COVID-19 Open Data

https://github.com/GoogleCloudPlatform/covid-19-open-data

## Selected Data

The project will primarily use the Google COVID-19 Open Data
epidemiology, vaccination and hospitalization tables.

## Data Handling

Raw datasets are intentionally not stored in GitHub because of
their size.

The raw data will be downloaded separately and uploaded to Hadoop HDFS
during the ingestion phase.

## HDFS Target

/covid/raw/
```

The Google repository documents these separate tables and their location/date-based structure. ([github.com](https://github.com/GoogleCloudPlatform/covid-19-open-data))

Step 10 — Create a project configuration file

Create:

```
config/project_config.yaml
```

Example:

YAML

```
project:
  name: covid-big-data-analytics
  version: 1.0

cluster:
  master: master
  workers:
    - worker1
    - worker2

hdfs:
  raw_path: /covid/raw
```

```
processed_path: /covid/processed
output_path: /covid/output
```

```
mongodb:
  database: covid_analytics
  collections:
    - country_summary
    - daily_summary
    - vaccination_summary
    - hospitalization_summary
```

Don't put passwords or connection strings containing credentials here.

Step 11 — First meaningful commit — YOU

Now check:

Bash

```
git status
```

You should see your new files.

Add:

Bash

```
git add .
```

Commit:

Bash

```
git commit -m "Initialize project architecture"
```

Push:

Bash

```
git push origin main
```

Your first faculty-visible contribution

```
Fenil
└─ Initialize project architecture
```

This is a **meaningful commit**.

Step 12 — Give Member 2 a real task

Don't let Member 2 just edit the README.

Give them:

Create the cluster and infrastructure documentation structure.

They should create:

```
docs/cluster/  
scripts/cluster/  
scripts/hdfs/
```

and add:

```
docs/cluster/cluster-requirements.md
```

Example:

Markdown

```
# 3-Node Hadoop Cluster  
  
## Cluster Architecture  
  
Master Node:  
- NameNode  
- ResourceManager  
  
Worker Node 1:  
- DataNode  
- NodeManager  
  
Worker Node 2:  
- DataNode  
- NodeManager  
  
## Minimum Requirement  
  
1 Master + 2 Workers
```

Then:

Bash

```
git add .  
git commit -m "Add Hadoop cluster documentation structure"  
git push
```

Faculty now sees:

```
Student 2  
└─ Add Hadoop cluster documentation structure
```

Step 13 — Give Member 3 a real task

Give Member 3:

Create the analytics and MongoDB project structure.

They create:

```
src/mongodb/  
src/analytics/  
notebooks/  
docs/analytics/
```

Add:

```
docs/analytics/analytics-plan.md
```

Example:

Markdown

```
# Analytics Plan  
  
## Planned Analytics  
  
1. Country-wise confirmed cases  
2. Country-wise deaths  
3. Daily new cases  
4. Top affected locations  
5. Vaccination analysis  
6. Hospitalization analysis  
7. Case fatality indicator
```

Then:

Bash

```
git add .  
git commit -m "Define analytics and MongoDB structure"  
git push
```

Now:

```
Student 3  
└─ Define analytics and MongoDB structure
```

Step 14 — Check everyone's GitHub history

Go to:

GitHub → Repository → Insights → Contributors

You should eventually see all three contributors.

At this stage, something like:

Contributor	Commits
You	1
Student 2	1
Student 3	1

Later we want:

You	10+
Student 2	7+
Student 3	7+

Don't artificially create commits just to increase the number.

Step 15 — Branch strategy

Since you're the leader, I recommend a simple branch strategy.

```
main  
|  
└─ feature/hdfs-cluster  
|
```

```
|— feature/spark-processing
|
|— feature/mongodb-analytics
```

Each member works on their feature branch.

For example, Member 2:

Bash

```
git checkout -b feature/hdfs-cluster
```

Work → commit → push:

Bash

```
git add .
git commit -m "Add HDFS cluster configuration"
git push -u origin feature/hdfs-cluster
```

Then create a Pull Request.

You review it and merge it into `main`.

This is much closer to an **industry Git workflow** than everyone directly pushing everything into `main`.

Phase 1 GitHub contribution map

Student	Ownership	Phase-1 contribution
You	Lead / Architecture	Repository + architecture + README
Member 2	Cluster/Data Engineering	Cluster documentation + infrastructure structure
Member 3	Analytics/NoSQL	Analytics plan + MongoDB structure

What NOT to do

Don't do this

```
You:
10 commits

Member 2:
"Update README"

Member 3:
"Update README"
```

Faculty may see that the work isn't genuinely distributed.

Don't do this either

```
You create everything
      ↓
Zip project
      ↓
Upload once
```

Your faculty specifically says this will not be accepted.

Instead

```
Issue
  ↓
Student develops feature
  ↓
Commit
  ↓
Pull Request
  ↓
Review
  ↓
Merge
```

Phase 1 final checklist

Before moving to Phase 2, verify:

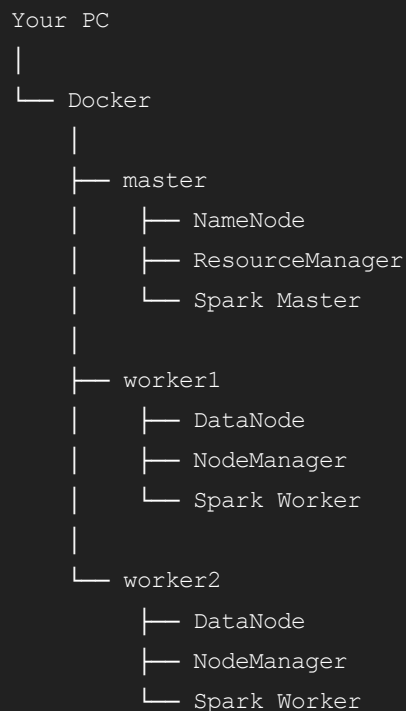
- ☒ GitHub repository created
- ☒ Repository is accessible to faculty
- ☒ 3 students added as collaborators
- ☒ README.md created
- ☒ .gitignore created
- ☒ requirements.txt created
- ☒ Project folders created

- ✓ Dataset README created
- ✓ Cluster documentation created
- ✓ Analytics plan created
- ✓ At least 1 meaningful commit from each student
- ✓ Everyone uses their own GitHub account
- ✓ No raw dataset committed
- ✓ No passwords/API keys committed

Then we move to Phase 2

Phase 2 = Set up the actual 3-node cluster.

I recommend we do that next as a **Docker-based cluster**:



That will give you a **reproducible 3-node environment** that can be stored in your GitHub repository and demonstrated to your faculty.